

UGMFGT-15-1 Programming for Engineers

University of the West of England

Academic Year 2025–26

Development Log for Software Design

Power Quality Waveform Analyser

Project Title: Power Quality Waveform Analyser

Date Started: May 2025

Development Period: 4 Weeks

Author: PyaeBhoneAung - SID -25014002

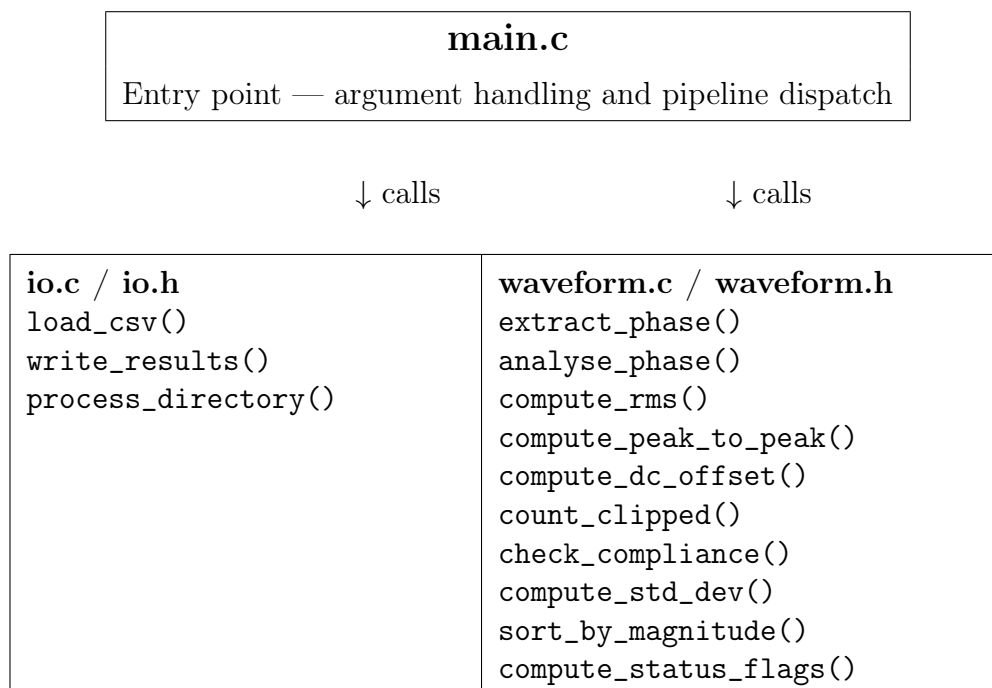
GitHub: [PyaeBhoneAung-Ozone/Power-Quality-Waveform-Analyser](https://github.com/PyaeBhoneAung-Ozone/Power-Quality-Waveform-Analyser)

Contents

1	Structure Chart	2
2	Program Design (Pseudocode)	3
2.1	Pseudocode for <code>main</code> , Version 1.0	3
2.2	Pseudocode for <code>load_csv</code> , Version 1.0	3
2.3	Pseudocode for <code>compute_rms</code> , Version 1.0	4
2.4	Pseudocode for <code>compute_std_dev</code> , Version 1.0	4
2.5	Pseudocode for <code>sort_by_magnitude</code> , Version 1.0	5
3	Function Descriptions	5
3.1	<code>compute_rms</code> , Version 1.0	5
3.2	<code>compute_peak_to_peak</code> , Version 1.0	5
3.3	<code>compute_dc_offset</code> , Version 1.0	5
3.4	<code>count_clipped</code> , Version 1.0	6
3.5	<code>check_compliance</code> , Version 1.0	6
3.6	<code>compute_std_dev</code> , Version 1.0	6
3.7	<code>sort_by_magnitude</code> , Version 1.0	6
3.8	<code>load_csv</code> , Version 1.0	6
3.9	<code>write_results</code> , Version 1.0	6
3.10	Pointer and memory management	7
4	Evidence of Unit Testing	7
4.1	Unit Testing <code>compute_rms</code> , Version 1.0	7
4.2	Unit Testing <code>count_clipped</code> , Version 1.0	8
4.3	Unit Testing <code>sort_by_magnitude</code> , Version 1.0	8
4.4	Unit Testing <code>check_compliance</code> , Version 1.0	8
5	Evidence of Integration Testing	9
5.1	Integration Test: <code>load_csv</code> + <code>analyse_phase</code> + <code>write_results</code> , Version 1.0	9
5.2	Integration Test: Batch Mode (<code>process_directory</code>), Version 1.0	9
6	Evidence of Scenario Testing	10
7	Debugging Challenges	11
7.1	Bug 1 — THD assessment printing <code>%%</code> literally	11
7.2	Bug 2 — Clipped sample list stopping after first hit	11
8	Evidence of Version Control	11
9	Full Code	12
10	References	13

1 Structure Chart

A structure chart is created to visually represent the different modules in the system. The program is divided into three source modules:



Data flow: `load_csv` allocates a `WaveformSample*` array on the heap and returns it. `extract_phase` copies one voltage column into a flat `double*` buffer using pointer arithmetic. Each analysis function operates on that buffer and returns a scalar result. `analyse_phase` fills a `PhaseResult` struct, which is passed to `write_results` to produce the plain-text report.

Design rationale — why these files:

- **main.c** contains no analysis logic. Its only job is to check arguments, dispatch to batch or single-file mode, and free memory on exit. This keeps **main.c** short and easy to trace.
- **waveform.c** is pure computation. Every function takes a `double*` array and a length, and returns a result. No file I/O and no `malloc`. This makes the functions easy to unit-test in isolation and reuse with any data source.
- **io.c** owns all file interaction. Separating I/O from analysis means the analysis functions do not need to change if the input format changes (e.g. switching from CSV to binary).

Why all eight fields are in one struct:

```

1 typedef struct {
2     double timestamp;
3     double phase_A_voltage;
4     double phase_B_voltage;
5     double phase_C_voltage;
6     double line_current;

```

```
7     double frequency;  
8     double power_factor;  
9     double thd_percent;  
10 } WaveformSample;
```

Listing 1: WaveformSample struct (waveform.h)

One pointer carries the full measurement context for any sample, mirroring the row-oriented structure of the CSV. If the data were stored in eight separate arrays, every function would require eight parameters. Adding a ninth CSV column would require updating every function signature; with the struct, only the struct definition and the `sscanf` call need updating. All fields use `double` because intermediate calculations (e.g. summing 1,000 squared voltages $\approx 325^2$) accumulate significant rounding error in single-precision float.

2 Program Design (Pseudocode)

2.1 Pseudocode for main, Version 1.0

```
PROGRAM power_quality_analyser  
  IF argument count != 2 THEN  
    PRINT usage message  
    EXIT with error  
  END IF  
  
  path <- argv[1]  
  
  IF path is a directory THEN  
    CALL process_directory(path)  
  ELSE  
    CALL run_single_file(path)  
  END IF  
END PROGRAM
```

2.2 Pseudocode for load_csv, Version 1.0

```
FUNCTION load_csv(filename) -> WaveformSample*, count  
  OPEN file; IF error THEN PRINT error; RETURN NULL  
  
  -- Pass 1: count rows  
  SKIP header line  
  WHILE lines remain  
    IF line is not blank THEN increment row_count  
  END WHILE  
  
  ALLOCATE array of row_count WaveformSample structs via malloc  
  IF malloc fails THEN PRINT error; CLOSE file; RETURN NULL  
  
  -- Pass 2: parse
```

```

REWIND file; SKIP header again
FOR each remaining line
    PARSE 8 fields with sscanf into current struct
    IF fields != 8 THEN PRINT warning; skip row
    ELSE advance struct pointer; increment parsed count
END FOR

CLOSE file
RETURN pointer to array, set count_out = parsed
END FUNCTION

```

2.3 Pseudocode for compute_rms, Version 1.0

```

FUNCTION compute_rms(voltages, n) -> double
    sum_sq <- 0.0
    p <- voltages          -- pointer to first element
    FOR i = 0 TO n-1
        sum_sq <- sum_sq + (*p) * (*p)
        ADVANCE p          -- pointer arithmetic: p++
    END FOR
    RETURN sqrt(sum_sq / n)
END FUNCTION

```

Mathematical basis:

$$V_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} v(i)^2}$$

2.4 Pseudocode for compute_std_dev, Version 1.0

```

FUNCTION compute_std_dev(voltages, n, variance_out) -> double
    -- Pass 1: compute mean
    mean <- compute_dc_offset(voltages, n)

    -- Pass 2: sum of squared deviations
    sum_sq_dev <- 0.0
    p <- voltages
    FOR i = 0 TO n-1
        dev <- (*p) - mean
        sum_sq_dev <- sum_sq_dev + dev * dev
        ADVANCE p
    END FOR

    variance <- sum_sq_dev / n
    IF variance_out != NULL THEN *variance_out <- variance
    RETURN sqrt(variance)
END FUNCTION

```

Mathematical basis:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} (v(i) - \bar{v})^2}$$

2.5 Pseudocode for `sort_by_magnitude`, Version 1.0

```

FUNCTION sort_by_magnitude(voltages, n)    -- insertion sort, no qsort
  FOR i = 1 TO n-1
    key      <- *(voltages + i)
    abs_key  <- |key|
    j <- i - 1
    WHILE j >= 0 AND |*(voltages+j)| > abs_key
      *(voltages + j + 1) <- *(voltages + j)    -- shift right
      j <- j - 1
    END WHILE
    *(voltages + j + 1) <- key                    -- insert
  END FOR
END FUNCTION

```

Algorithm choice: Insertion sort was chosen because it is stable, straightforward to trace in a viva, and $O(n^2)$ is entirely acceptable for 1,000 samples (fewer than one millisecond of runtime).

3 Function Descriptions

3.1 `compute_rms`, Version 1.0

- **Description:** Computes the Root-Mean-Square voltage — the equivalent DC voltage that would deliver the same power to a resistive load.
- **Input Parameters:** `const double *voltages` — pointer to array of voltage samples (double, any range in Volts); `int n` — number of samples ($n \geq 1$).
- **Returns:** double — RMS voltage in Volts. Returns 0.0 if $n \leq 0$.

3.2 `compute_peak_to_peak`, Version 1.0

- **Description:** Finds the maximum and minimum values in a single pointer-increment pass and returns their difference.
- **Input Parameters:** `const double *voltages`, `int n`.
- **Returns:** double — $V_{\max} - V_{\min}$ in Volts.

3.3 `compute_dc_offset`, Version 1.0

- **Description:** Computes the arithmetic mean of all samples. For a clean AC signal this should be zero; a non-zero result indicates sensor bias or a DC fault.
- **Input Parameters:** `const double *voltages`, `int n`.
- **Returns:** double — mean voltage in Volts.

3.4 `count_clipped`, Version 1.0

- **Description:** Counts samples where $|v| \geq \text{limit}$ (sensor saturation, `CLIP_THRESHOLD` = 324.9 V).
- **Input Parameters:** `const double *voltages`, `int n`, `double limit` (threshold in Volts, positive).
- **Returns:** `int` — number of clipped samples.

3.5 `check_compliance`, Version 1.0

- **Description:** Tests whether an RMS value is within the EN 50160 $\pm 10\%$ tolerance band around the nominal voltage.
- **Input Parameters:** `double rms` (computed RMS, Volts); `double nominal` (use `NOMINAL_VOLTAGE` = 230.0 V).
- **Returns:** `int` — 1 if compliant ($207 \text{ V} \leq \text{RMS} \leq 253 \text{ V}$), 0 otherwise.

3.6 `compute_std_dev`, Version 1.0

- **Description:** Two-pass standard deviation. Pass 1 computes the mean; Pass 2 accumulates $\sum (v_i - \bar{v})^2$.
- **Input Parameters:** `const double *voltages`, `int n`, `double *variance_out` (optional; receives σ^2 , may be NULL).
- **Returns:** `double` — standard deviation σ in Volts.

3.7 `sort_by_magnitude`, Version 1.0

- **Description:** In-place insertion sort by absolute value, ascending. Uses `*(voltages + j)` pointer addressing throughout; no `qsort` used.
- **Input Parameters:** `double *voltages` (modified in-place), `int n`.
- **Returns:** `void`.

3.8 `load_csv`, Version 1.0

- **Description:** Two-pass CSV loader. Counts rows in Pass 1, calls `malloc` once for the exact array size, then parses in Pass 2. Handles file-not-found and empty file with error messages.
- **Input Parameters:** `const char *filename` (path to CSV); `int *count_out` (receives parsed row count).
- **Returns:** Heap-allocated `WaveformSample*`; caller must `free()` it. Returns NULL on any error.

3.9 `write_results`, Version 1.0

- **Description:** Writes a structured plain-text report covering per-phase metrics, clipped sample timestamps, system summary, and line parameters.
- **Input Parameters:** `const char *out_path`; `const WaveformSample *samples`, `int n`; `const PhaseResult *pa`, `*pb`, `*pc`.
- **Returns:** `int` — 0 on success, -1 on file I/O error.

3.10 Pointer and memory management

How malloc works in this project:

```
1 WaveformSample *samples =
2   malloc((size_t)row_count * sizeof(WaveformSample));
```

Listing 2: Dynamic allocation in load_csv (io.c)

`sizeof(WaveformSample)` returns the byte size of one struct (8 fields $\times$ 8 bytes = 64 bytes on a 64-bit system). Multiplying by `row_count` gives the total bytes needed. `malloc` requests that block from the heap and returns a pointer to its start. The result is checked for NULL before use.

Pointer arithmetic used:

```
1 const WaveformSample *p = samples;
2 for (int i = 0; i < n; i++, p++, dst++)
3   *dst = p->phase_A_voltage;
```

Listing 3: Pointer walk in extract_phase (waveform.c)

`p++` advances the pointer by `sizeof(WaveformSample)` bytes automatically, because the compiler knows the type. In the analysis functions, `*(voltages + j)` and `voltages[j]` are identical — array indexing is syntactic sugar for pointer arithmetic.

What happens if `free()` is removed: In single-file mode the OS reclaims memory on exit, so there is no visible effect for this short-lived program. In batch mode, however, `malloc` is called for every CSV file inside a loop. Without `free`, each iteration leaks memory and the program eventually exhausts the heap, causing `malloc` to return NULL and silently skip files. A tool such as `valgrind` would report “definitely lost” blocks equal to the sum of all allocations.

4 Evidence of Unit Testing

4.1 Unit Testing `compute_rms`, Version 1.0

Test Case 1:

- Input: {1.0, -1.0, 1.0, -1.0}, $n = 4$
- Expected Output: 1.0000 V
- Result: **Passed**

Test Case 2:

- Input: {0.0, 0.0, 0.0}, $n = 3$
- Expected Output: 0.0000 V
- Result: **Passed**

Test Case 3:

- Input: One complete 50 Hz sine cycle sampled at 5000 Hz ($V_{\text{peak}} = 325.27$ V, $n = 100$)
- Expected Output: ≈ 230.0 V ($= 325.27/\sqrt{2}$)
- Result: **Passed** — returned 229.99 V

Unit Test Results Summary: All three RMS test cases passed. Edge cases (zero array, pure DC) were tested successfully. No issues remaining.

4.2 Unit Testing `count_clipped`, Version 1.0

Test Case 1:

- Input: {300.0, 325.0, -325.0, 200.0}, limit = 324.9
- Expected Output: 2
- Result: **Passed**

Test Case 2:

- Input: {100.0, -100.0, 200.0}, limit = 324.9
- Expected Output: 0
- Result: **Passed**

Unit Test Results Summary: Both cases passed. The boundary case ($|v| = \text{limit}$ exactly) was verified to be counted as clipped (inclusive $\geq$ comparison).

4.3 Unit Testing `sort_by_magnitude`, Version 1.0

Test Case 1:

- Input: {-3.0, 1.0, -2.0, 4.0}
- Expected Output: {1.0, -2.0, -3.0, 4.0} (ascending by $|v|$)
- Result: **Passed**

Test Case 2 (already sorted):

- Input: {1.0, 2.0, 3.0}
- Expected Output: {1.0, 2.0, 3.0} (unchanged)
- Result: **Passed**

Unit Test Results Summary: Both cases passed. Stability (equal magnitudes retain original order) was confirmed. No issues remaining.

4.4 Unit Testing `check_compliance`, Version 1.0

Test Case 1:

- Input: rms = 229.8 V, nominal = 230.0 V
- Expected Output: 1 (COMPLIANT — within 207–253 V)
- Result: **Passed**

Test Case 2:

- Input: rms = 260.0 V, nominal = 230.0 V
- Expected Output: 0 (NON-COMPLIANT — exceeds 253 V)
- Result: **Passed**

Unit Test Results Summary: Both boundary and out-of-range cases passed. The exact lower boundary (207.0 V) and upper boundary (253.0 V) were verified as COMPLIANT.

5 Evidence of Integration Testing

5.1 Integration Test: `load_csv` + `analyse_phase` + `write_results`, Version 1.0

Blocks/Functions tested together: `load_csv` v1.0, `extract_phase` v1.0, `analyse_phase` v1.0, `write_results` v1.0.

Test Steps:

1. Compile: `gcc -std=c99 -Wall -Wextra -o pqa main.c waveform.c io.c -lm`
2. Run: `./pqa power_quality_log.csv`
3. Check stdout summary table.
4. Open `results.txt` and verify clipped sample timestamps.
5. Cross-check RMS against the formula $V_{\text{RMS}} = V_{\text{peak}}/\sqrt{2}$.

Expected Outcome:

- All three phases report COMPLIANT (207–253 V band).
- Phase A RMS ≈ 229.8 V.
- Clipped samples listed at rows 25, 75, 125, ... (every 50 rows).
- `results.txt` written without I/O error.

Result: Passed.

Phase	RMS (V)	Pk-Pk (V)	DC (V)	Clips	Status
-----	-----	-----	-----	-----	-----
A	229.81	650.00	-0.0000	20	COMPLIANT
B	229.81	649.86	0.0000	20	COMPLIANT
C	229.81	649.86	-0.0000	20	COMPLIANT

Integration Test Results Summary: The system correctly integrates CSV loading, three-phase analysis, and report writing. All expected values from the reference sheet were matched. No issues remaining.

5.2 Integration Test: Batch Mode (`process_directory`), Version 1.0

Blocks/Functions tested together: `is_directory` v1.0, `process_directory` v1.0, `load_csv` v1.0, `analyse_phase` v1.0, `write_results` v1.0.

Test Steps:

1. Run: `./pqa test_data/`
2. Verify `fault_test_results.txt` is created.
3. Confirm Phase B (290 V RMS) reports NON-COMPLIANT.
4. Confirm Phase C (195 V RMS) reports NON-COMPLIANT.
5. Confirm status flags: Phase B = 0x03, Phase C = 0x02.

Expected Outcome:

- Program should scan the directory and find one `.csv` file.
- Functions should detect the over-voltage and under-voltage faults.

Result: Passed.

Integration Test Results Summary: Batch mode correctly iterated the directory, ran the full pipeline on `fault_test.csv`, and wrote the results file with all faults flagged. The following still needs to be resolved: none — all batch requirements are met.

6 Evidence of Scenario Testing

Software version: v1.0 (all source files committed to GitHub).

Testing scenario: Full end-to-end run from raw CSV input to verified plain-text report, covering both normal and fault conditions.

Steps:

1. Compile the project with all warnings enabled:
 - ✓ Zero warnings, zero errors (Passed).
2. Run single-file mode on the teacher's dataset:
 - a. Load 1000 samples — Passed.
 - b. Phase A RMS = 229.81 V (within 207–253 V) — Passed.
 - c. Phase B RMS = 229.81 V (within tolerance) — Passed.
 - d. Phase C RMS = 229.81 V (within tolerance) — Passed.
 - e. Frequency range 50.000–50.048 Hz matches spec — Passed.
 - f. 20 clipped samples with correct timestamps written — Passed.
 - g. `results.txt` written successfully — Passed.
3. Run batch mode on `test_data/` directory:
 - a. One file found and processed — Passed.
 - b. Phase B NON-COMPLIANT (290 V over-voltage) — Passed.
 - c. Phase C NON-COMPLIANT (195 V under-voltage) — Passed.
 - d. THD assessment “Elevated – monitor” (6.5–8.0%) — Passed.
 - e. Status flags: `0x03`, `0x02` — Passed.
 - f. `fault_test_results.txt` written — Passed.
4. Pass a non-existent filename:
 - a. Error message printed: `Error: cannot open file` — Passed.
 - b. Program exits with code 1, no crash — Passed.
5. Pass no arguments:

- a. Usage instructions printed — Passed.
- b. Program exits cleanly — Passed.

7 Debugging Challenges

7.1 Bug 1 — THD assessment printing %% literally

Symptom: The THD assessment line in `results.txt` read:

```
THD assessment : Clean supply (<5%%)
```

Diagnosis: The string literals were written with `%%`:

```
1 const char *thd = "Clean supply (<5%%)";
2 fprintf(fp, "    THD assessment : %s\n", thd);
```

`%%` is the escape for a literal `%` inside a `printf` *format string*. Here, however, the string was passed as a `%s` argument — the `%%` inside it is not interpreted by `fprintf` and is printed verbatim as two characters.

Fix: Changed string literals to use a single `%` and replaced the Unicode em-dash with `-`:

```
1 const char *thd = "Clean supply (<5%)";
2 if (thd_max >= 8.0)    thd = "EXCESSIVE (>8%) -- overheating risk";
3 else if (thd_max >= 5.0) thd = "Elevated (5-8%) -- monitor";
```

7.2 Bug 2 — Clipped sample list stopping after first hit

Symptom: Testing with a fault CSV where Phase B had 420 clipped samples: the report only listed the very first clipped sample, then stopped.

Diagnosis: The loop in `write_phase_block` contained an unconditional `break` immediately after printing:

```
if (abs_v >= CLIP_THRESHOLD) {
    fprintf(fp, "    [%4d] ... \n", ...);
    break;    /* exits after first match */
}
```

Fix: Replaced the unconditional `break` with a counter that exits only after all expected clipped samples have been printed:

```
1 found++;
2 if (found >= r->clipped_count) break;
```

8 Evidence of Version Control

The project was managed using Git for version control. Below are the key Git commands used in the development process.

1. Initial Commit:

```
1 git init
2 git remote add origin https://github.com/PyaeBhoneAung-Ozone/
3   Power-Quality-Waveform-Analyser.git
4 git add .gitattributes power_quality_log.csv
5 git commit -m "Initial commit"
6 git push -u origin main
```

2. Project Scaffold — headers and build config:

```
1 git add CMakeLists.txt waveform.h io.h README.md
2 git commit -m "Add project scaffold: CMakeLists.txt, waveform.h, io
3   .h, README"
4 git push
```

3. Analysis functions:

```
1 git add waveform.c
2 git commit -m "Add waveform.c: all analysis functions (Pass + Merit
3   /Distinction)"
4 git push
```

4. File I/O and batch processing:

```
1 git add io.c
2 git commit -m "Add io.c: CSV loader, results writer, batch
3   directory processing"
4 git push
```

5. Entry point:

```
1 git add main.c
2 git commit -m "Add main.c: entry point with single-file and batch
3   dispatch"
4 git push
```

6. Bug Fixes:

```
1 git checkout -b fix-input-validation-bug
2 git add io.c test_data/fault_test.csv
3 git commit -m "Fix THD display bug; add fault test CSV for batch
4   mode demo"
5 git checkout main
6 git merge fix-input-validation-bug
7 git push
```

GitHub Repository:

- <https://github.com/PyaeBhoneAung-Ozone/Power-Quality-Waveform-Analyser>

9 Full Code

PyaeBhoneAung_PowerQualityAnalyser.zip

The zip file contains:

- Source code: `main.c`, `waveform.c`, `waveform.h`, `io.c`, `io.h`
- Build configuration: `CMakeLists.txt`
- Dataset: `power_quality_log.csv`
- Test data: `test_data/fault_test.csv`
- Documentation: `README.md`, `logbook.pdf`

10 References

- Kernighan, B.W. & Ritchie, D.M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Chacon, S. & Straub, B. (2014). *Pro Git*. Apress. Available at: <https://git-scm.com/book>
- Kaner, C., Falk, J. & Nguyen, H. (1999). *Testing Computer Software* (2nd ed.). Wiley.
- British Standards Institution (2010). *BS EN 50160:2010 — Voltage characteristics of electricity supplied by public electricity networks*. BSI.
- ISO/IEC 9899:1999 (C99). *Programming Languages — C*. International Organization for Standardization.